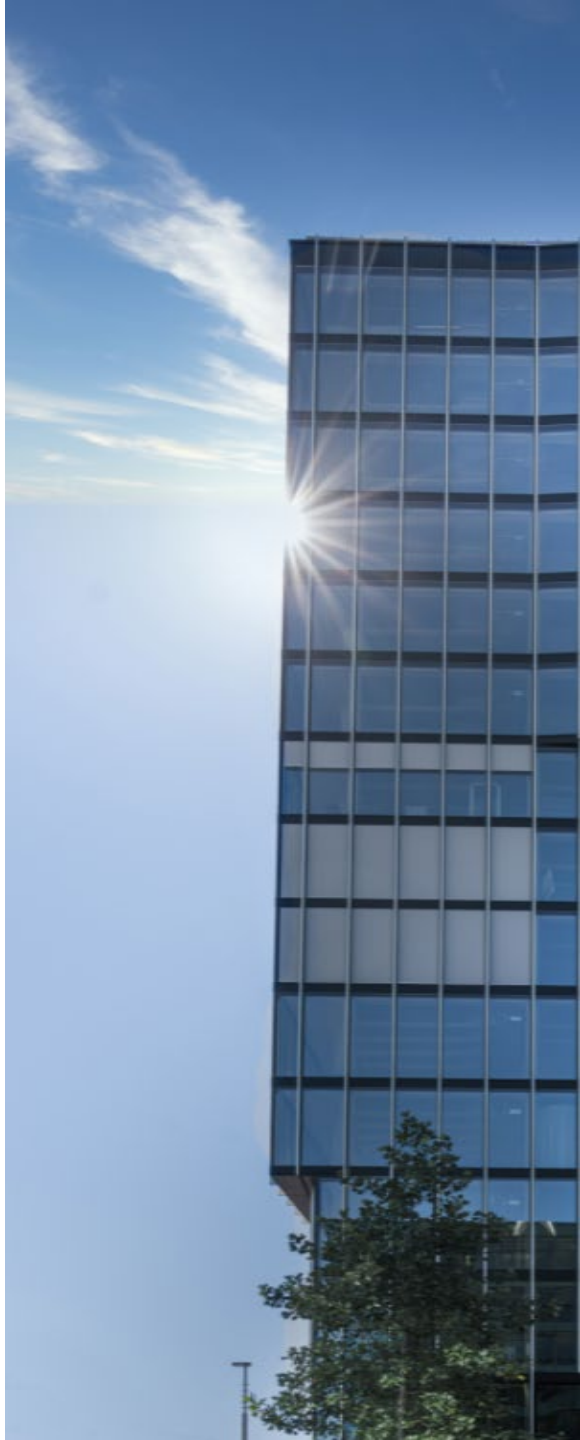
HSLU Hochschule Luzern

Objektorientierte Programmierung

`final`, `static` und Enumerationen

Roland Gisler



Inhalt

- Schlüsselwort `final`
- Schlüsselwort `static`
- Enumerationen (`enum`)

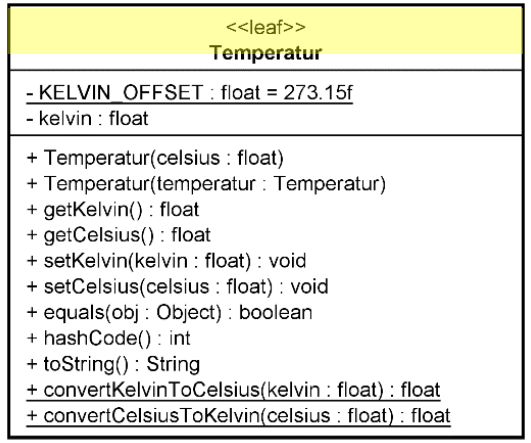
Lernziele

- Sie können das Schlüsselwort `final` in den verschiedenen Kontexten von Variablen, Methoden und Klassen sinnvoll anwenden.
- Sie kennen den Sinn und Zweck von mit `static` gekennzeichneten Attributen und Methoden.
- Sie können Enumerationen definieren und anwenden.

Schlüsselwort `final`

Schlüsselwort `final`

- Kann eingesetzt werden bei
  - lokalen Variablen und formalen Parametern,
  - Attributen und Methoden,
  - Klassen
- Bedeutet allgemein, dass etwas «unveränderbar» wird.
  - Exakte Bedeutung ist abhängig vom konkreten Einsatzort.
- Kann in UML-Diagrammen (leider) nicht überall visualisiert werden und wird darum leider auch sehr selten verwendet.
  - Der Stereotyp `<<leaf>>` signalisiert z.B. dass eine Klasse `final` ist und somit nicht mehr spezialisiert werden kann.



Finale lokale Variablen

- Beispiel: Zuweisung an eine lokale, **finale** Variable vom Typ `int`.

```
final int count = students.size();
```

- Die Variable **`count`** erhält die Anzahl der in der Collection **`students`** enthaltenen Elemente als Wert zugewiesen.
- `final` bewirkt, dass der Variablen **`count`** genau **nur einmal** ein Wert **zugewiesen** werden kann.
  - Der Wert ist danach **nicht** mehr veränderbar.
- Zweck: Man bewirkt (und signalisiert) damit, dass der Wert nach der initialen Zuweisung **konstant** bleibt.
  - Einsatz ist eine **Frage des Programmierstils**.

➔ Verhindert bei späteren Codeänderungen eventuell Fehler!

Finale formale Parameter

- Analog zu lokalen Variablen können auch formale Parameter als `final` deklariert werden. Beispiel:

```
public void setCelsius(final float celsius) { … }
```

- `final` bewirkt, dass der als aktueller Parameter übergebene Wert innerhalb der Methode **nicht** überschrieben werden kann.
- Zweck: Man bewirkt (und signalisiert) damit, dass der Parameter innerhalb der ganzen Methode **konstant** bleibt.
  - Einsatz ist eine **Frage des Programmierstils**.
  - Vorteil: Man hat die Sicherheit, dass der Originalwert in der **ganzen** Methode zur Verfügung steht.

➔ Verhindert bei späteren Codeerweiterungen eventuell Fehler!

Finale Attribute (auch ➔statisch möglich)

- Attribute einer Klasse als **`final`** markieren. Beispiel:

```
private final String uid;
```

- Finalen Attributen kann nur **einmalig** (pro Objekt) ein Wert zugewiesen werden:
  - Entweder direkt bei der Deklaration (seltener).
  - Oder einmalig innerhalb eines Konstruktors (häufiger).
- Zweck: Finale Attribute sind die Grundlage für **➔immutable Klassen** (bzw. zumindest ausgewählte Attribute).
  - Beispiel: **`User`**-Klasse mit **`uid`**, **`name`** und **`vorname`** als Attribute.
  - Die **`uid`** wird bei der Erstellung festgelegt und ist danach **unveränderlich**, während Namen und Vornamen noch verändert (mutiert) werden können.

➔ Finalisierte Attribute sind ein **essentieller Designentscheid**!

Finale Methoden

- Methoden können mit `final` markiert werden. Beispiel:

```
public final double getCelsius() { … }
```

- Finalisierte Methoden können in allfälligen Spezialisierungen (Vererbung) **nicht mehr überschrieben** (override) werden.
- Zweck: Damit schützt man Methoden in Spezialisierungen bewusst vor dem (un-)absichtlichen bzw. unerwünschten Überschreiben.
- Typische Beispiele bzw. Situationen:
  - Einfache Setter- und Getter-Methoden können finalisiert werden.
  - Abschliessend implementierte Methoden auf teilabstrakten Klassen können finalisiert werden.
  - Beispiel: `equals()` und `hashCode()` sollten sogar meistens finalisiert werden!

➔ Finalisierte Methoden sind ein **essentieller Designentscheid**!

Finale Klassen

- Klassen können mit `final` als finalisiert gekennzeichnet werden:

```
public final class Temperatur { … }
```

- Finalisierte Klassen können **nicht mehr spezialisiert** (vererbt) werden.
  - Alle enthaltenen Methoden sind damit implizit auch `final`.
- Zweck: Man verhindert damit explizit die Spezialisierung von Klassen, wenn sie dafür nicht vorgesehen oder ungeeignet sind.
  - Ein grundlegendes Designprinzip zur Vererbung sagt, dass Klassen entweder für Vererbung auszulegen sind oder man es andernfalls explizit verhindern soll. Darum: Finalisieren!

  ➔ Vererbung verursacht eine sehr hohe (stärkste) Kopplung!

➔Finalisierte Klassen sind ein **sehr essentieller Designentscheid**!

Empfehlungen zu `final`

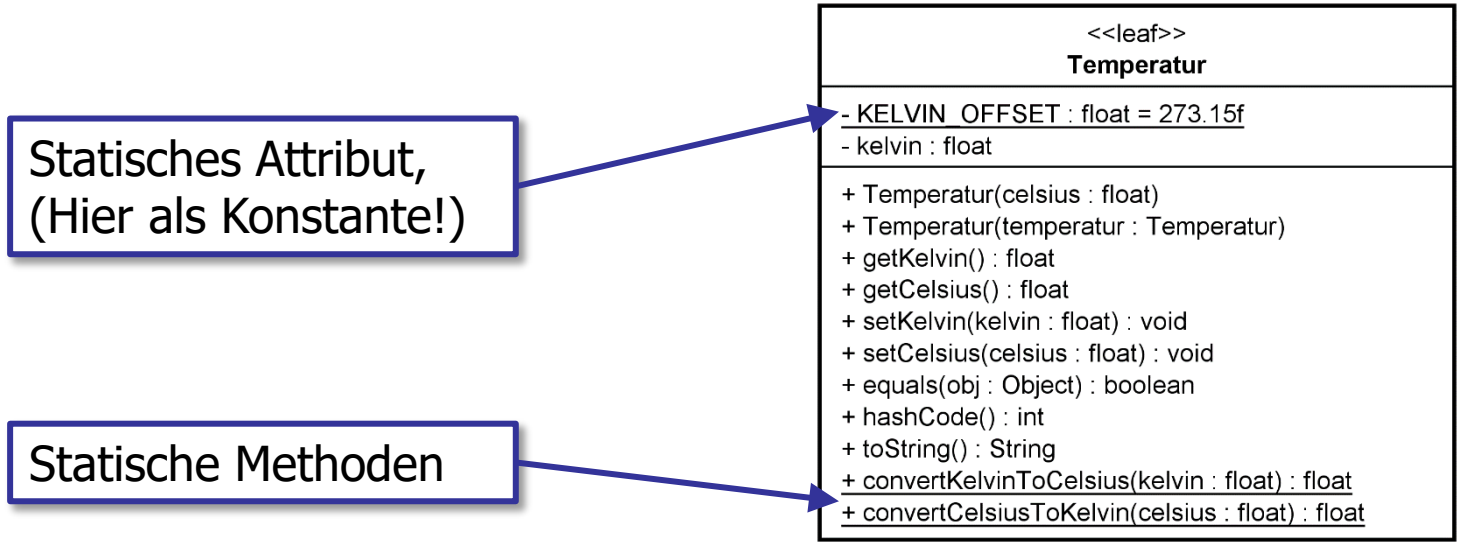
- Der Nutzen und Einsatz von `final` wird oft kontrovers diskutiert. Das liegt an der grossen Bandbreite der möglichen Einsatzbereiche!
- **Finale** lokale **Variablen** und formale **Parameter** sind stark vom individuellen **Programmierstil** abhängig und darum tatsächlich etwas **kontrovers**.
- **Finale Methoden** und **Klassen** sind aber (sehr) **essentielle Designentscheidungen**, welche die spätere Erweiterung und Veränderung eines OO-Designs nachhaltig beeinflussen können.
- Empfehlung: Analog zur Datenkapselung (so «privat» wie möglich, so «public» wie nötig) finalisieren Sie besser zu viel als zu wenig!
  - Etwas wieder veränderbar zu machen ist kein Problem, etwas nachträglich zu finalisieren hingegen fast unmöglich!

➔ Bei Klassen: Entweder explizit für Spezialisierung entwerfen oder dann `final`!

Schlüsselwort `static`

Schlüsselwort `static`

- Das Schlüsselwort **`static`** kann u.a. auf Attribute und Methoden angewendet werden.
- Statische Elemente können direkt über die Klasse referenziert und verwendet werden!
  - Somit also **ohne,** dass ein **Objekt** existiert.
  - Werden als Klassenvariablen bzw. Klassenmethoden bezeichnet.
  - Existieren pro Klasse **nur genau einmal**!
- In UML-Klassendiagrammen werden statische Elemente <u>unterstrichen</u> dargestellt!



Statische Attribute

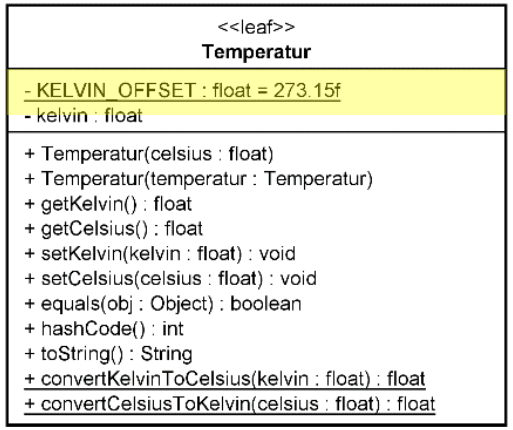
- Ein statisch deklariertes Attribut existiert nur genau **einmal** pro Klasse, und nicht **n**-mal pro Objekt!
  - Es wird gar kein Objekt (Instanz) benötigt!
- Ein statisches Attribut ist eine **veränderbare** Variable, welche aber nur einmal auf einer Klasse existiert.
- Beispiel:

```
private static int counter = 21;
```

- Statische Attribute werden (entsprechende Sichtbarkeit vorausgesetzt) direkt über den Klassennamen referenziert.
- Beispiel:

```
int newId = Person.counter++;
```

Statische Attribute als Konstante



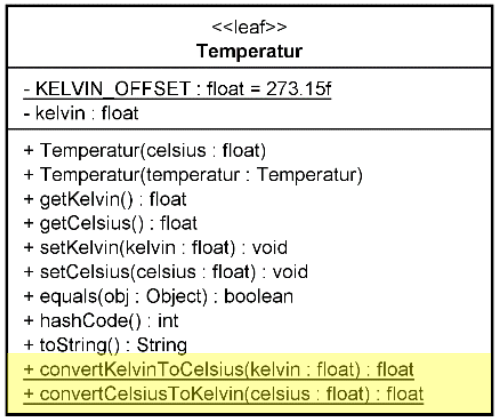
- Der häufigste Anwendungsfall: Ein statisches Attribut, welches gleichzeitig ➔finalisiert (`final`) ist und somit als **Konstante** dient.
  - Konstanten werden mit `GROSSEN_BUCHSTABEN` bezeichnet.
  - Wert kann (wegen `final`) nicht mehr verändert werden.
  - ➔ Häufigste Art der Verwendung von statischen Attributen!
- Beispiel eines statischen Attributes als Konstante:

```
public static final float KELVIN_OFFSET = 273.15f;
```

- Beispiel für die Verwendung:

```
float kelvin = celsius + Temperatur.KELVIN_OFFSET;
```

Statische Methoden



- Eine statische Methode kann direkt auf einer Klasse aufgerufen werden, **ohne** dass ein Objekt existiert.
- Statische Methoden sind sinnvoll, wenn eine Funktion **stateless**, d.h. **unabhängig** von Attributwerten (➔dem Objektzustand) ist.
- Beispiel:

```
public static float convertCelsiusToKelvin(float celsius) {
    ...
}
```

- Statische Methoden werden (entsprechende Sichtbarkeit vorausgesetzt) **immer** über den Klassennamen referenziert.
- Beispiel:

```
float kelvin = Temperatur.convertCelsiusToKelvin(20.0f);
```

Beispiele für spezielle statische Attribute

Folgende statische Attribute sind Ihnen sicher schon begegnet:

```
System.out.println("Hallo Welt!");
```

- Aufgrund der Schreibweise (**System**) erkennt man, dass es eine Klasse (nicht Objekt) ist, somit muss **out** ein statisches (und **public**!) Attribut sein.
  - Konkret: Ein Objekt vom Typ **PrintStream**, welches u.a. die Methode **println(…)** zur Verfügung stellt.

Math.PI *oder* **Math.E** *oder* **Math.sin(Math.toRadians(45));**

- Die Klasse **Math** enthält einige mathematische Konstanten und viele statische Methoden für mathematische Funktionen.
  - siehe JavaDoc: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Math.html

  ➔Es wäre ineffizient, immer ein Objekt erzeugen zu müssen!

Eine spezielle statische Methode

- Die folgende, spezielle, statische Methode ist Ihnen mit Sicherheit auch schon mehrfach begegnet:

```
public static void main(String[] args) {
    ...
}
```

- Die **`main(…)`**-Methode (mit **exakt** dieser Deklaration) repräsentiert eine Art «Vertrag»: Sie ist der Eintrittspunkt für den Start einer Klasse bzw. einer ganzen Java-Applikation!
  - Löst das «Huhn oder Ei»-Problem (zumindest für Java ☺).
- Die Java Virtual Machine (JVM) sucht, wenn wir ihr eine Klasse zum Starten übergeben, nach dieser Methode und führt sie aus.
  - Fehlt die **`main(…)`**-Methode, wird eine Fehlermeldung ausgegeben.

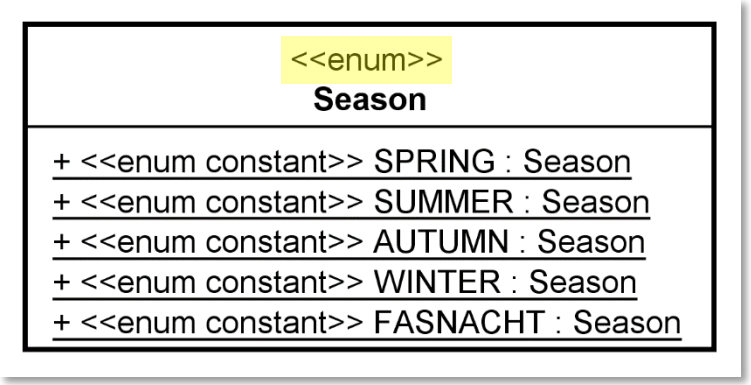
Enumerationen

Schlüsselwort `enum` (statt `class`)

- Mit einer Enumeration wird sowohl ein eigener Typ als auch eine Menge von logisch zusammenhängenden, konstanten Werten dieses Typs deklariert.

```
public enum Season {
    SPRING, SUMMER, AUTUMN, WINTER;
}
```

- Enumerationen sind
  - leicht verständlich,
  - sehr aussagekräftig und
  - **typsicher** ➔ Hauptvorteil!
- In Klassendiagrammen tragen sie den Stereotyp **`<<enum>>`**.



Enumerationen – Einsatz

- Enumerationen können somit wie jede andere Klasse als Typ verwendet und eingesetzt werden. Beispiel:

```
Season current = Season.AUTUMN;
```

  - Können auch in einem `switch`-Statement verwendet werden.

- Enumerationen sind **normale Klassen**, mit **vier** Ausnahmen:
  - Werden mit dem Schlüsselwort `enum` deklariert (statt `class`).
  - Sind implizit **finalisiert** (können nicht spezialisiert werden).
  - Enthalten die einzelnen Enum-Werte implizit als **finale** und **statische** Klassenattribute,
  - welche **automatisch** (über privaten Konstruktor) instanziiert werden.

- Enumerationen ersetzten numerische Codierungskonstanten!

Enumerationen - mit Attributen und Methoden

- Enumerationen können, weil sie eigentlich (fast) normale Klassen sind, auch beliebige, zusätzliche Attribute und Methoden erhalten. Beispiel:

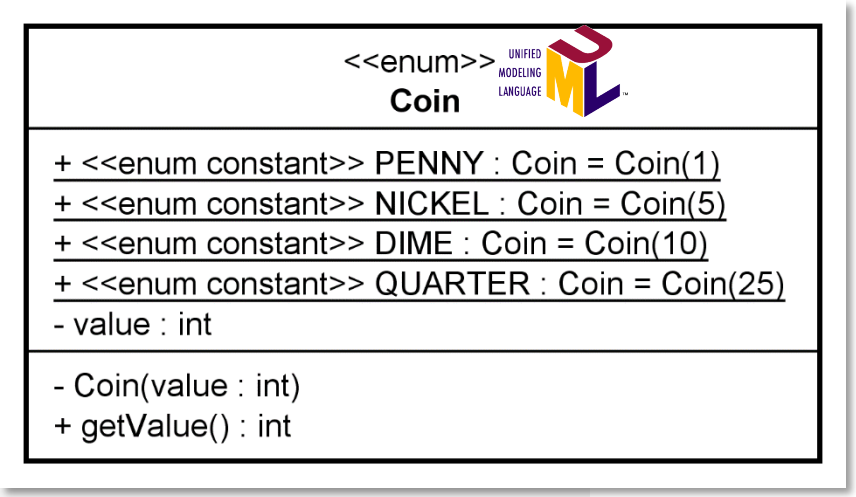
```java
public enum Coin {

    PENNY(1), NICKEL(5), DIME(10), QUARTER(25);

    private final int value;

    private Coin(final int value) {
        this.value = value;
    }

    public int getValue() {
        return this.value;
    }
}
```



Empfehlungen zu Enumerationen

- `toString()` soll für einen Enumerationswert eine `String`-Repräsentation, z.B. für Persistierung oder Logging liefern.
- `valueOf(…)` soll zu einer `String`-Repräsentation wiederum den passenden Enumerationswert liefern.
- Enumerationen sind ein **typsicheres** und sehr **elegantes** Mittel, sie sollten überlegt, aber tendenziell häufig(er) eingesetzt werden.
  ➔ Nutzen und Eleganz von Enumerationen wird oft unterschätzt.
- Enumerationen wurden in Java 1.5 ergänzt.
  - Früher: `public static final int`-Orgien auf Klassen.
  - Ein wirklich schlimmes Anti-Pattern: `static final`-Attribute die auf einem Interface definiert werden.

  ➔Unbedingt Refactoring in Betracht ziehen!

Hintergrundinformationen zu Enumerationen

- Die Definition einer Enum entspricht eigentlich nichts anderem als der Instanzierung von Enum-Objekten in statischen und finalen Variablen mit den Namen der verschiedenen Enum-Werte!
  - Dafür wird je implizit der (private) Konstruktor aufgerufen.
  - Wollen wir zusätzliche Attributwerte definieren, müssen wir dafür also einen entsprechenden Konstruktor ergänzen.
- Jedem Enum-Wert wird vom Compiler ein sogenannter Ordinal (eindeutiger `int`-Wert) zur schnellen Identifikation zugewiesen.
  - Diese Werte können zwar abgefragt werden, sollten aber **niemals** verwendet werden, weil sie bei der Veränderung einer Enum (und Neukompilation) **nicht stabil** bleiben!

➔ Methode `ordinal()` ist somit **tabu** und keinesfalls für eine Persistierung geeignet!

Zusammenfassung

- Schlüsselwort `static`
  - Statische (Klassen-)Variablen, häufig für Konstanten verwendet.
  - Statische (Klassen-)Methoden, ohne Instanzierung aufrufbar.
  - Start einer Java-Applikation:

```
public static void main(String[] args)
```

- Schlüsselwort `final`
  - Für finale (lokale) Variablen.
  - Für finale formale Parameter.
  - Für finale (auch statisch) Attribute.
  - Für finale, nicht mehr überschreibbare Methoden.
  - Für finale, nicht mehr spezialisierbare Klassen.
- Enumerationen als spezielle Klassen für Aufzählungstypen.
  - Eigener Typ, implizit `final` und somit nicht spezialisierbar.

Fragen?